

Training Metrics Reference

What Every Logged Number Means

Pulak Mehrotra

April 2026

Contents

1	Primary Metric: Pooling Savings	1
2	Reward Signals	2
3	SAC Internals	4
4	Environment Metrics	7
5	Throughput	8
6	Augmentation Metrics	8
7	Fault Diagnosis Table	8
8	Reward Function Design — Best Practices	9
8.1	Align the reward with the true objective, not a proxy	9
8.2	Potential-based reward shaping (PBRs)	9
8.3	Do not mix incommensurable objectives with a fixed λ	9
8.4	Reward hacking and specification gaming	10
8.5	Sparse terminal reward with short sub-episodes	10
8.6	Optimal-gap reward shaping via <code>find_optimal</code>	11
8.7	Behavioral cloning pre-training from the optimal solver	11
8.8	Difference rewards for topology-aware credit assignment	12
8.9	Summary: proposed reward redesign	13

All metrics are logged to TensorBoard under `output/logs/<run-id>/`. View with `tensorboard --logdir output/logs/`. Note: W&B was disabled (`--wandb` not passed) for runs 5–7; all plots are TensorBoard only. To enable W&B for future runs add `--wandb` to the training command.

1 Primary Metric: Pooling Savings

`eval/pooling_savings_mean` is the number that matters. It is computed as:

$$\text{pooling_savings} = 1 - \text{pooling_ratio} \tag{1}$$

$$\text{pooling_ratio} = \frac{\max_j c_j \cdot N_{\text{MPD}}}{D_{\text{pod}}} \tag{2}$$

where c_j is the peak load on MPD j , N_{MPD} is the number of MPDs in the pod, and D_{pod} is the total DRAM of the pod.

Intuitively, the pooling ratio measures how much of the shared MPD capacity the agent wastes by concentrating load. A ratio of 1.0 means no savings over a non-pooled baseline; a ratio of 0.8 means 20% savings.

- **savings** > 0: agent improves on baseline. Run 5 peaked at ≈ 0.013 –0.020.
- **savings** = 0: equivalent to no pooling.
- **savings** < 0: agent is actively harmful—worse than not pooling. Run 6 hit -0.62 due to a collapsed policy.

`env/pooling_ratio` and `env/pooling_savings` are the training-environment versions computed from a single episode mid-training—much noisier. `eval/pooling_savings_mean` is cleaner: 10 random pod mappings on the held-out eval trace.

2 Reward Signals

`eval/mean_reward`

Mean episode reward on the eval trace (from SB3’s `EvalCallback`). This is the raw reward sum, not pooling savings. These two variants are *not* comparable in magnitude — run 6’s -244 vs. run 7’s -25 does **not** mean run 7 is $10\times$ better.

Shared notation

Throughout this section, let:

- t — current tick (5-minute resolution); VM v arrives at tick t
- i — host receiving the arriving VM
- $\mathcal{N}(i)$ — set of MPDs reachable by host i (from the CXL topology)
- c_j — current load on MPD j (GB), before this allocation
- c_j^{post} — load on MPD j after this allocation is applied
- D_{pod} — total DRAM of the pod (GB); used as a normaliser
- $\bar{c} = D_{\text{pod}}/N_{\text{MPD}}$ — fair-share load per MPD (GB)
- λ — penalty weight for secondary terms (`variance_lambda` or `reward_lambda`)

Departure relief $D_j(t, W)$

Variants A and B share a time-weighted departure-relief signal $D_j(t, W)$ that approximates how much load on MPD j will drain within the next W ticks (default $W = 200$ ticks ≈ 16.7 hours).

$$D_j(t, W) = \sum_{\substack{v \in \text{active}(j) \\ \text{end}(v) \leq t+W}} \text{mem}(v) \cdot \left(1 - \frac{\text{end}(v) - t}{W}\right) \quad (3)$$

where $\text{active}(j)$ is the set of VMs currently allocated on MPD j , and $\text{mem}(v)$ is the GB allocated to VM v on MPD j .

The weight $(1 - (\text{end}(v) - t)/W)$ is largest for VMs departing soon and falls to zero for VMs departing exactly at $t + W$. VMs departing after $t + W$ contribute nothing.

Intuition. A VM departing in 10 minutes counts almost fully toward relief; a VM with three days remaining is treated as sticky load and contributes zero. Two MPDs at identical current load c_j can therefore have very different projected loads after subtracting D_j .

Projected sticky load. Define the normalised projected load on MPD j after the current allocation:

$$\hat{c}_j^+(t) = \frac{c_j^{\text{post}} - D_j(t, W)}{D_{\text{pod}}} \quad (4)$$

This is the fraction of D_{pod} that MPD j will still be carrying once near-term departures drain. A high $\hat{c}_j^+$ means the MPD will remain a bottleneck; a low value means load will drain soon.

Reward variant A (runs 5, 7)

Variant A penalises the worst *projected sticky load* among the MPDs this host can directly reach:

$$r_t^A = - \max_{j \in \mathcal{N}(i)} \hat{c}_j^+(t) \quad (5)$$

Every reachable MPD contributes signal at every step, not just the pod-wide worst. This makes the reward dense: bad placements onto any reachable MPD cause an immediate penalty. Typical episode sum: -30 to -20 .

Why this is better than the “current” variant. r_t^A scores allocations against durable load $(c_j^{\text{post}} - D_j)$ rather than the raw post-allocation peak. A placement onto a full-but-draining MPD is penalised less than a placement onto a full-and-sticky MPD, because D_j subtracts the relief that is already coming.

Reward variant B

Variant B extends A with a penalty for hot MPDs that the current host *cannot* reach. Those MPDs are shared with neighboring hosts, so overloading them harms future placements that those neighbors will attempt.

Let $\bar{\mathcal{N}}(i) = \{1, \dots, N_{\text{MPD}}\} \setminus \mathcal{N}(i)$ be the set of MPDs unreachable by host i . Define the normalised pre-allocation projected load on an unreachable MPD:

$$P_j(t) = \frac{c_j - D_j(t, W)}{D_{\text{pod}}}, \quad j \notin \mathcal{N}(i) \quad (6)$$

Note that c_j here is the *pre*-allocation load (unreachable MPDs are unchanged by this step). Then:

$$r_t^B = r_t^A - \lambda \cdot \max_{j \in \bar{\mathcal{N}}(i)} P_j(t) \quad (7)$$

with $\lambda = 0.2$ by default (`reward_lambda`).

The global term discourages a host from saturating shared MPDs even when its own reachable set looks comfortable, capturing indirect congestion effects that r_t^A alone cannot see.

Reward variant “current” (run 6)

The original reward penalises the *increase in raw peak* normalized by fair share, plus a variance penalty over all MPD loads:

$$r_t^{\text{cur}} = -\frac{c_{\max}^{\text{post}} - c_{\max}}{\bar{c}} - \lambda \cdot \text{Var}\left(\frac{c_j}{\bar{c}}\right) \quad (8)$$

$$\text{where } c_{\max} = \max_j c_j, \quad c_{\max}^{\text{post}} = \max_j c_j^{\text{post}}, \quad \bar{c} = \frac{D_{\text{pod}}}{N_{\text{MPD}}}$$

with $\lambda = 0.5$ (`variance_lambda`).

Why this variant failed (run 6).

- **Single-MPD signal.** $c_{\max}^{\text{post}} - c_{\max}$ is nonzero only when the allocation lands on the *current* worst MPD or creates a new worst MPD. All other placements return $r = -\lambda \cdot \text{Var}(\cdot)$, which is the same regardless of which MPD is chosen — giving the policy almost no gradient for distinguishing good from bad placements on non-worst MPDs.
- **No departure model.** c_j treats a VM departing in 10 minutes identically to one with 3 days remaining. The policy has no signal to prefer draining MPDs.
- **Result.** SAC collapsed all allocations onto one or two MPDs; max/min load ratio $\sim 20\times$; greedy outperformed the learned policy.

Comparison

Variant	Signal scope	Departure-aware	Key failure mode
current	Single worst MPD	No	Degenerate: all load collapses to one MPD
A	All reachable MPDs	Yes (D_j)	Blind to shared-MPD congestion
B	All MPDs (local + global)	Yes (D_j)	None identified yet; ablation pending

`eval/mean_ep_length`

Average episode length in VM arrival events. Typical: 6,000–8,000. Very low values suggest trace loading or HOTFIX filter issues.

3 SAC Internals

What SAC is doing

Soft Actor–Critic (SAC) [1] maximises a reward objective augmented with a policy-entropy bonus:

$$J(\pi) = \mathbb{E}_{\tau \sim \pi} \left[\sum_{t=0}^T \gamma^t \left(r_t + \alpha \mathcal{H}(\pi(\cdot | s_t)) \right) \right] \quad (9)$$

where $\mathcal{H}(\pi(\cdot | s)) = -\mathbb{E}_{a \sim \pi}[\log \pi(a | s)]$ is the Shannon entropy of the policy at state s , $\alpha > 0$ is the *entropy coefficient* (temperature), and $\gamma = 0.999$ in this project.

The entropy term keeps the policy stochastic: it is rewarded for spreading probability mass across good actions rather than collapsing to a single deterministic choice. Without it, the policy converges prematurely to a local optimum.

Figure 1 shows the components. All five interact at every gradient update.

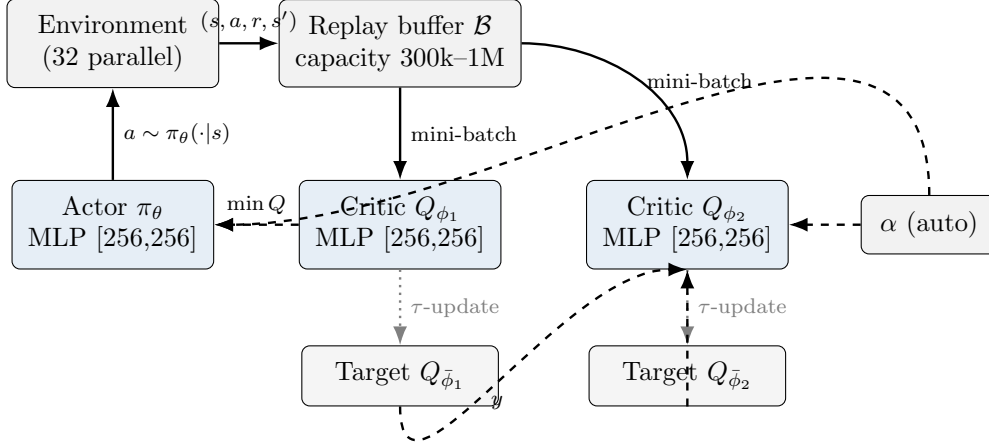


Figure 1: SAC architecture as used in this project. Solid arrows: data flow during training (mini-batches from replay buffer). Dashed: gradient-coupled terms. Dotted: Polyak target-network update ($\tau = 0.005$). The two critics independently estimate Q ; their minimum is used everywhere to suppress overestimation.

Critic update — Bellman residual

SAC maintains two independent critic networks Q_{ϕ_1}, Q_{ϕ_2} (the “clipped double-Q” trick from TD3 [2]) to suppress Q-value overestimation.

At each gradient step, a mini-batch $(s, a, r, s') \sim \mathcal{B}$ is drawn from the replay buffer. The *Bellman target* is computed using the frozen target networks $Q_{\bar{\phi}_1}, Q_{\bar{\phi}_2}$:

$$y = r + \gamma \underbrace{\left(\min_{i=1,2} Q_{\bar{\phi}_i}(s', \tilde{a}') - \alpha \log \pi_\theta(\tilde{a}' | s') \right)}_{\text{soft value of next state } V(s')}, \quad \tilde{a}' \sim \pi_\theta(\cdot | s') \quad (10)$$

The subtracted $\alpha \log \pi$ term is the entropy bonus: the policy is credited for being uncertain about the next action. The min across the two target critics suppresses over-optimistic bootstrapping.

Both critics are then updated by minimising mean-squared Bellman error:

$$\mathcal{L}_{\text{critic}} = \mathbb{E}_{(s,a,r,s') \sim \mathcal{B}} \left[\frac{1}{2} \sum_{i=1}^2 \left(Q_{\phi_i}(s, a) - y \right)^2 \right] \quad (11)$$

Project values. $\gamma = 0.999$, `batch_size` = 256 (fast: 512), `learning_starts` = 1,000 (no updates until buffer has at least 1k transitions).

Actor update — entropy-regularised policy gradient

The actor is updated to maximise Q-value while also maximising entropy. Concretely, the actor loss is:

$$\mathcal{L}_{\text{actor}} = \mathbb{E}_{s \sim \mathcal{B}, \tilde{a} \sim \pi_{\theta}(\cdot | s)} \left[\alpha \log \pi_{\theta}(\tilde{a} | s) - \min_{i=1,2} Q_{\phi_i}(s, \tilde{a}) \right] \quad (12)$$

Minimising $\mathcal{L}_{\text{actor}}$ does two things simultaneously:

- The $-Q$ term pushes the actor toward actions with high Q-value (exploit what the critics have learned).
- The $+\alpha \log \pi$ term penalises deterministic policies (low-entropy distributions have high $\log \pi$ under the chosen action).

Note: the Q-values used here are from the *live* critics ϕ_i , not the frozen targets. Gradients flow through π_{θ} but not Q_{ϕ_i} .

What the logged metric means. `sac/actor_loss` = the value of $\mathcal{L}_{\text{actor}}$ at the last gradient step. It becomes more negative as the policy improves (higher Q-values dominate). A *rising* or *flat* actor loss with bad pooling savings usually means the critic has converged to accurately predict the value of a *bad* policy — the actor is optimising toward the wrong target.

Automatic entropy tuning (α)

SAC automatically tunes α so that the policy maintains a target entropy $\mathcal{H}_{\text{target}}$. SB3 defaults this to $-\dim(\mathcal{A})$, the negative of the action dimension. The loss for α is:

$$\mathcal{L}_{\alpha} = \mathbb{E}_{s \sim \mathcal{B}, \tilde{a} \sim \pi_{\theta}(\cdot | s)} \left[-\alpha (\log \pi_{\theta}(\tilde{a} | s) + \mathcal{H}_{\text{target}}) \right] \quad (13)$$

If the policy’s entropy $-\log \pi$ falls *below* $\mathcal{H}_{\text{target}}$, the gradient increases α (more exploration pressure). If entropy is already above target, α decreases. α is stored as $\log \alpha$ internally to keep it positive; `sac/ent_coef` reports $\exp(\log \alpha)$.

- **Healthy:** 0.1–1.0, declining slowly as the policy specialises.
- **Collapsed** ($\alpha < 0.01$): policy became deterministic too early, stuck in a local optimum. Run 6 hit $\alpha = 0.00079$ at 560k steps — the policy was packing all load onto one MPD with near-zero variance in its action.
- **Too high** ($\alpha > 2$): too much noise, policy not committing to good allocations.

Target network update (Polyak averaging)

After every gradient step the target critics $\bar{\phi}_i$ are moved slightly toward the live critics:

$$\bar{\phi}_i \leftarrow \tau \phi_i + (1 - \tau) \bar{\phi}_i \quad (14)$$

with $\tau = 0.005$ (`tau`). The target networks lag behind the live critics, preventing the Bellman target y from moving too fast and causing divergence. The smaller τ is, the more stable but slower the bootstrap.

Metric-by-metric reference

sac/q1_mean, sac/q2_mean. Mean of $Q_{\phi_1}(s, a)$ and $Q_{\phi_2}(s, a)$ sampled from the replay buffer. Should rise as the policy improves (higher expected discounted return). If they grow without bound the critics are overestimating — the min trick only suppresses this partially. The two critics should track each other: large systematic divergence means one critic is miscalibrated.

sac/q_spread. $\mathbb{E}|Q_{\phi_1}(s, a) - Q_{\phi_2}(s, a)|$ from a sampled mini-batch. This measures critic disagreement. Values below 0.1 mean the critics agree; above 0.5 suggests the value landscape is shifting faster than the critics can track.

sac/critic_loss. $\mathcal{L}_{\text{critic}}$ (Eq. 11) at the last gradient step. Should decrease over training. **Caution:** near-zero critic loss with bad pooling savings means the critics have accurately learned the value of a *bad* policy — not that the policy is good. This is the most common misleading signal in run 6.

sac/actor_loss. $\mathcal{L}_{\text{actor}}$ (Eq. 12). Becomes more negative as Q-values rise. A flat or rising actor loss with stable critic loss is the classic sign that the actor is stuck at a local optimum the critics have accurately valued.

sac/actor_grad_norm, sac/critic_grad_norm. ℓ_2 norm of all gradient tensors in the actor and critic networks respectively, computed as $\|\nabla\|_2 = \sqrt{\sum_p \|\nabla_p\|_2^2}$.

- Critic norm rising sharply (e.g. 65 in run 7 late stage): large Bellman errors, value landscape changing fast; consider reducing `learning_rate`.
- Any norm > 100 : potential instability; gradient clipping may help.
- Actor grad norm $\rightarrow 0$: actor not updating, effectively frozen. Check entropy collapse.

train/n_updates. Cumulative gradient steps taken. Each step draws one mini-batch and runs one pass through Eqs. 11–13. The update rate is determined by `train_freq` and `n_envs`:

$$\text{updates} = \frac{\text{total_timesteps}}{\text{train_freq} \times \text{n_envs}} \quad (15)$$

Run 7 (`train_freq=8`, `n_envs=32`, 2M steps): $2,000,000 / (8 \times 32) = 7,812$ updates — exactly what was logged. Run 6 (`train_freq=1`, `n_envs=8`): $\sim 70\text{k}$ updates at 560k steps, which is why it was $10\times$ slower per wall-clock minute.

4 Environment Metrics

env/max_peak

Peak MPD load (GB) seen in the last training episode. Absolute value depends on the pod’s total DRAM. Useful for checking trace diversity across envs.

env/num_events

Number of VM arrival events in the episode. Varies by pod and trace. Typical: 5,000–10,000.

5 Throughput

time/fps

Timesteps per second (SB3 “frames per second”), including env step, policy inference, and gradient update.

Run	fps	n_envs	Cause of deviation
Run 5	244	32	Healthy baseline
Run 7	52	32	CPU contention (run 6 running in parallel)
Run 6	22	8	train_freq=1, gradient updates dominating

6 Augmentation Metrics

Only logged when `--augmentation` is enabled (runs 5 and 7).

aug/scale_mean, aug/scale_std

Distribution of memory scaling factors applied to episodes. Should stay near $[0.7, 1.2]$ with mean ≈ 0.95 .

aug/link_failures_applied

Count of episodes with random topology link failures. Zero in runs 5/7 (`aug_link_failures=0.0`).

aug/unique_traces

Number of distinct traces seen in the last logging window. With `multi_trace=True` and 10 traces, expect ~ 10 after warmup.

7 Fault Diagnosis Table

Symptom	Likely cause
<code>pooling_savings</code> < 0 <code>ent_coef</code> $\rightarrow 0$ early	Policy collapsed; always allocates to one MPD. Over-updating (<code>train_freq</code> too low) or poorly tuned entropy target.
<code>q1_mean</code> exploding	Q-value overestimation; try lower learning rate or smaller network.
<code>critic_loss</code> ≈ 0 but bad savings	Critic converged to accurately predict a bad policy’s values.
<code>fps</code> $\ll$ expected	CPU contention; too many parallel envs for available cores.
<code>pooling_savings_std</code> high	Policy inconsistent across pod mappings; may need more training or augmentation.
Actor grad norm $\rightarrow 0$ Critic grad norm $\gg 50$	Actor stuck; policy not improving. Value landscape shifting fast; consider lower learning rate or grad clipping.

8 Reward Function Design — Best Practices

The results across runs 5–7 show a consistent ceiling of $\sim 1\text{--}1.6\%$ pooling savings, policy entropy collapse by 100k steps, and no clear improvement from longer training. These are all symptoms of a misdesigned reward function. This section documents best practices from recent ML literature and maps them directly onto the failure modes observed here.

8.1 Align the reward with the true objective, not a proxy

The most dangerous mistake in reward design. Our reward measures Δ_{peak} (immediate MPD load change per step), but the true objective is *episode-level pooling savings*—a quantity that only manifests at the end of 7,000+ steps. These diverge because:

- An agent can minimise Δ_{peak} at every step by spreading load uniformly, even when this creates future fragmentation that prevents large VMs from fitting efficiently.
- The D_j lookahead term attempts to compensate by encoding near-future departures, but as the *Noisy Future Knowledge* analysis (Uzeel, 2025, internal) shows: if each allocation decision is off by even 5%, compounded predictions over 8 days do *worse than greedy*. This exactly explains the observed savings ceiling.

Fix direction. Use the episode-level pooling savings as the terminal reward, with *potential-based shaping* (Section 8.2) as the only source of dense intermediate signal. This is theoretically grounded and cannot change the optimal policy [4].

8.2 Potential-based reward shaping (PBRs)

Ng, Harada, and Russell (1999) [4] proved that an agent’s set of optimal policies is preserved *if and only if* shaping rewards take the form:

$$F(s, s') = \gamma \Phi(s') - \Phi(s) \tag{16}$$

where $\Phi : \mathcal{S} \rightarrow \mathbb{R}$ is an arbitrary *potential function*. Any other dense reward formulation risks making a suboptimal allocation globally optimal under the shaped objective.

Our current reward is *not* in this form— $(\Delta_{\text{peak}}/\bar{c})$ adds information about the transition that is not expressible as a potential difference, so it can (and does) induce a different optimal policy than the true objective.

Valid potential for this system. A natural choice is the negative normalised peak:

$$\Phi(s) = -\frac{\max_j c_j}{D_{\text{pod}}} \tag{17}$$

Then $F(s, s') = \gamma\Phi(s') - \Phi(s)$ rewards reductions in the peak MPD load in a policy-invariant way. Combined with the sparse terminal reward (true pooling savings), this provides dense learning signal without distorting the objective. Recent work [5] shows a linear shift of Φ can improve convergence speed without changing the encoded preference.

8.3 Do not mix incommensurable objectives with a fixed λ

The **current** reward variant (Eq. 8) mixes two terms: peak reduction and load variance, with $\lambda = 0.5$ fixed for all of training. This is problematic on two levels:

1. **The right trade-off changes during training.** Early in training, the policy is random and load variance is high; λ should down-weight the variance term. Later, once peak is controlled, variance matters more. A fixed scalar cannot adapt.
2. **The variance term is semantically wrong for this topology.** Equal load across MPDs is impossible—some MPDs serve more hosts and will always carry more traffic. Penalising variance rewards the policy for replicating a uniform distribution that the topology does not support.

Principled alternatives. Multi-reward RL literature [8] recommends:

- **Dynamic weight scheduling:** adjust λ via a bandit algorithm based on which objective is most violated at each stage.
- **Lexicographic reward:** first optimise the primary objective (peak); use the secondary (variance) only as a tiebreaker when peak is tied.
- **Reward decomposition:** train separate value heads per objective and combine at the policy level [9].

8.4 Reward hacking and specification gaming

A dense per-step reward creates strong pressure to find specification loopholes. DeepMind’s survey [7] documents hundreds of cases; the relevant pattern here is *proxy gaming*:

“The agent optimises the measured metric without fulfilling the underlying goal. Higher model capacity amplifies the divergence.”

In our system the agent discovered by $\sim 100k$ steps that spreading load uniformly minimises both Δ_{peak} and variance—scoring well on the reward without exploiting the pooling topology at all. This is exactly specification gaming: the metric is satisfied, the goal is not.

Diagnostic signal. If `eval/pooling_savings_mean` plateaus while `train/critic_loss` continues falling, the agent is gaming the proxy. The critic is learning an accurate value function for a suboptimal policy.

8.5 Sparse terminal reward with short sub-episodes

The noisy-future-knowledge result suggests a structural fix: shorten the episode so prediction errors do not compound. The RL equivalent of the *disjoint sliding window* (Uzeel, 2025) is:

- Break the 14-day trace into **1–2 day sub-episodes**.
- Give the terminal reward (true pooling savings) at the end of each sub-episode, not across the full trace.
- Use PBRS shaping within the sub-episode for dense signal.

This reduces compounding from $\sim 7,000$ steps to ~ 500 – $1,000$ steps, keeping prediction-based signals accurate enough to be useful. The technique is standard in systems RL (Pensieve [10] uses per-chunk rewards; Decima [11] uses job-completion rewards rather than per-step queue-length penalties).

8.6 Optimal-gap reward shaping via `find_optimal`

The codebase includes an exact per-snapshot solver (`octopus/optimal.py`) that computes the minimum achievable peak MPD load for a given vector of host demands and topology M . It uses binary search over a Dinic max-flow feasibility check and returns $t^*(s_t)$ in $O(n \cdot m \cdot \sqrt{n+m} \cdot \log(1/\epsilon))$ time per call. This scalar is a lower bound on any policy’s peak load at the current state — it is the best any allocator could do with full knowledge of instantaneous demands and the topology, but *no* knowledge of future arrivals.

Optimality-gap penalty. Define the normalised optimality gap after the current allocation:

$$\delta_t = \frac{c_{\max}^{\text{post}} - t^*(s_t)}{D_{\text{pod}}} \quad (18)$$

$\delta_t = 0$ means the agent’s placement is as good as optimal given current demands. $\delta_t > 0$ quantifies how much peak capacity the agent wasted. Add this as a shaping term:

$$r'_t = r_t^A - \beta \delta_t, \quad \beta \in [0.1, 0.5] \quad (19)$$

Unlike reward variants A/B, this term is grounded in a provably tight lower bound rather than a heuristic departure estimate.

Potential-based version (policy-invariant). To guarantee the shaped reward does not change the set of optimal policies (Section 8.2), define the potential:

$$\Phi(s) = -\frac{t^*(s)}{D_{\text{pod}}} \quad (20)$$

Then $F(s, s') = \gamma\Phi(s') - \Phi(s)$ rewards reductions in the optimal lower bound itself. Combined with the sparse terminal reward, this is theoretically the safest dense signal available: it pushes the policy toward states where the optimal achievable peak is low, without encoding any assumption about departures or future arrivals.

Implementation note. `find_optimal` runs a max-flow per call and is not free. At 1,000 steps/episode $\times$ 32 envs, calling it every step would dominate wall-clock time. Practical options:

- Call every $k = 10\text{--}50$ steps and hold the last value constant between calls (stale-optimal shaping).
- Call only at episode end to compute a terminal potential bonus: $\Phi(s_T) = -t^*(s_T)/D_{\text{pod}}$.
- Pre-compute t^* offline for a representative grid of demand vectors and use nearest-neighbour lookup at training time.

8.7 Behavioral cloning pre-training from the optimal solver

The SAC policy collapse observed in run 6 occurs during early training, when the replay buffer contains only random-policy transitions and entropy collapses by $\sim 100\text{k}$ steps. If the actor starts near a reasonable allocation policy instead of random, entropy collapse is less likely and the critic learns from meaningful transitions from the first update.

`find_optimal` returns the optimal *peak scalar* t^* , not allocation weights directly. However, the Dinic max-flow solution that achieves t^* encodes a feasible flow assignment: the edge flows f_{ij} from host i to MPD j give the optimal memory split. Extracting these and normalising per host yields expert allocation weights $a_j^* = f_{ij} / \sum_j f_{ij}$.

Pre-training procedure.

1. For a large set of pod snapshots sampled from the training traces, call the extended solver to extract (s, a^*) pairs.
2. Pre-train the actor π_θ via supervised behavioural cloning (BC):

$$\mathcal{L}_{\text{BC}} = \mathbb{E}_{(s, a^*) \sim \mathcal{D}_{\text{opt}}} \left[\|\pi_\theta(s) - a^*\|_2^2 \right] \quad (21)$$

3. Initialise SAC with the pre-trained actor weights; proceed with standard SAC training. The critics are still initialised randomly and learn from scratch via the Bellman update.

Why this works. BC loss is minimised when the actor outputs the flow-optimal allocation weights. The actor starts training already able to spread load near-optimally for instantaneous demands. SAC then fine-tunes it to account for long-horizon effects (VM lifetimes, future arrivals) that the myopic solver ignores.

Limitations. The optimal solver is myopic: it does not know that a VM arriving on MPD j right now will still be there in 48 hours. The pre-trained actor inherits this blindness. SAC’s long-horizon critic must correct for it. This is a feature, not a bug — BC warm-starts the actor at “instantaneously optimal”, and RL improves it toward “temporally optimal”.

Alternative: DAgger-style action blending. Rather than a separate pre-training phase, blend the optimal action and the policy action throughout early training:

$$a_t^{\text{exec}} = (1 - \lambda_t) \pi_\theta(s_t) + \lambda_t a_t^*, \quad \lambda_t = \max\left(0, 1 - \frac{t}{T_{\text{anneal}}}\right) \quad (22)$$

λ_t anneals from 1 (fully expert) to 0 (fully learned) over T_{anneal} steps. The replay buffer stores $(s_t, a_t^{\text{exec}}, r_t, s_{t+1})$ so the critic learns from blended actions. This is simpler to implement than a separate BC phase but mixes the learning signal.

8.8 Difference rewards for topology-aware credit assignment

In placement problems with a graph topology, per-step rewards give poor *credit assignment*—the agent cannot tell which of its many past decisions caused a bad episode outcome. The systems multi-agent RL literature recommends **difference rewards** [12]:

$$r_i = G(\text{joint allocation}) - G(\text{joint allocation without agent } i) \quad (23)$$

Applied here: the reward for placing a VM on MPD j is the counterfactual improvement in final pooling savings over a greedy baseline placement. This requires a fast baseline rollout but gives the agent a tight, unambiguous signal for each individual decision.

8.9 Summary: proposed reward redesign

Current design	Problem	Proposed fix
Per-step Δ peak proxy	Agents game the proxy; savings plateau at 1.5%	Sparse terminal pooling savings
D_j lookahead in reward	Compounds noise over 7,000 steps; worse than greedy	Move to PBRS potential only
Fixed λ peak+variance	Wrong trade-off, wrong topology assumption	Lexicographic or dynamic weighting
14-day episodes	Prediction errors compound; sparse terminal signal	1–2 day sub-episodes
No counterfactual baseline	Poor credit assignment across 7k steps	Difference rewards vs. greedy
No lower bound on achievable peak	Reward gives no signal about how far from optimal each placement is	Optimal-gap shaping δ_t or PBRS with $\Phi(s) = -t^*(s)/D_{\text{pod}}$
Random actor initialisation	Entropy collapse by 100k steps; critic learns value of a bad policy	BC pre-train from flow-optimal weights, or DAgger blending

References

- [1] T. Haarnoja, A. Zhou, P. Abbeel, S. Levine. Soft Actor-Critic: Off-Policy Maximum Entropy Deep Reinforcement Learning with a Stochastic Actor. In *ICML*, 2018.
- [2] S. Fujimoto, H. van Hoof, D. Meger. Addressing Function Approximation Error in Actor-Critic Methods. In *ICML*, 2018.
- [3] A. Raffin, A. Hill, A. Gleave, A. Kanervisto, M. Ernestus, N. Dormann. Stable-Baselines3: Reliable Reinforcement Learning Implementations. *JMLR*, 22(268):1–8, 2021.
- [4] A. Ng, D. Harada, S. Russell. Policy Invariance Under Reward Transformations: Theory and Application to Reward Shaping. In *ICML*, 1999. <https://www.semanticscholar.org/paper/Policy-Invariance-Under-Reward-Transformations:-and-Ng-Harada/94066dc12fe31e96af7557838159bde598cb4f10>
- [5] Improving the Effectiveness of Potential-Based Reward Shaping in Reinforcement Learning. arXiv, 2025. <https://arxiv.org/html/2502.01307v1>
- [6] Comprehensive Overview of Reward Engineering and Shaping in Advancing Reinforcement Learning Applications. arXiv, 2024. <https://arxiv.org/html/2408.10215v1>
- [7] V. Krakovna et al. Specification Gaming: the Flip Side of AI Ingenuity. DeepMind Blog, 2020. <https://deepmind.google/blog/specification-gaming-the-flip-side-of-ai-ingenuity/>
- [8] Reward Models in Deep Reinforcement Learning: A Survey. *IJCAI*, 2025. <https://www.ijcai.org/proceedings/2025/1199.pdf>
- [9] Multi-Reward Reinforcement Learning — EmergentMind topic overview, 2025. <https://www.emergentmind.com/topics/multi-reward-reinforcement-learning-rl>

- [10] H. Mao et al. Real World Performance of Adaptive Bitrate Algorithms (Pensieve). In *ACM SIGCOMM*, 2017. <https://dl.acm.org/doi/10.1145/3098822.3098843>
- [11] H. Mao et al. Learning Scheduling Algorithms for Data Processing Clusters (Decima). In *ACM SIGCOMM*, 2019. <https://dl.acm.org/doi/10.1145/3341302.3342080>
- [12] D. H. Wolpert, K. Tumer. Collective Intelligence, Data Routing and Braess' Paradox. *Journal of Artificial Intelligence Research*, 16:359–387, 2002. <https://www.jair.org/index.php/jair/article/view/10256>